Orbs: Architecture and an Open Implementation

How Amp's orbs work inside, how we self-hosted them on exe.dev, and how moonlet rebuilds them on pi-durable

swair · October 2026

1 The Idea
- 1.1 Vocabulary

2 Amp's Architecture
- 2.1 Provider and Jobs
- 2.2 Bootstrap
- 2.3 The Executor
- 2.4 Setup
- 2.5 Lifecycle

3 What Orbs Support

4 Self-Hosting on exe.dev

5 Moonlet
- 5.1 Control Plane
- 5.2 The Pipe
- 5.3 Orb Lifecycle
- 5.4 Setup
- 5.5 Dispatch
- 5.6 Security

6 Results

7 Open Questions

An orb is a remote machine that belongs to one agent thread. The agent works there instead of on the user's laptop; the machine sleeps when the thread goes idle and wakes when the next message arrives. This paper describes Amp's implementation as observed from inside a live self-hosted orb, records what it took to run Amp's orbs on exe.dev, and presents moonlet, an open implementation in which a pi-durable Harness replaces Amp's thread actor and pi-env's daemon replaces Amp's in-orb executor, while the machine-provisioning contract stays byte-for-byte compatible with Amp's.

1. The Idea

An orb separates three things that a local coding agent keeps in one process: the conversation, the model calls, and the machine on which tools run. The conversation and the model calls live on a server. The machine is disposable compute that can be paused for weeks at no cost and resumed in seconds with its files, and ideally its processes, intact. The agent therefore keeps working when the laptop is closed, many agents can run at once, and each has a whole machine to install dependencies, start services and prove its changes.

Self-hosting changes who owns the machine but not the shape of the system. Amp states the requirement precisely: the infrastructure must start machines quickly, put them to sleep and resume them equally fast, keep them for a long time, and scale to zero so sleeping machines cost nothing. Everything else is protocol.

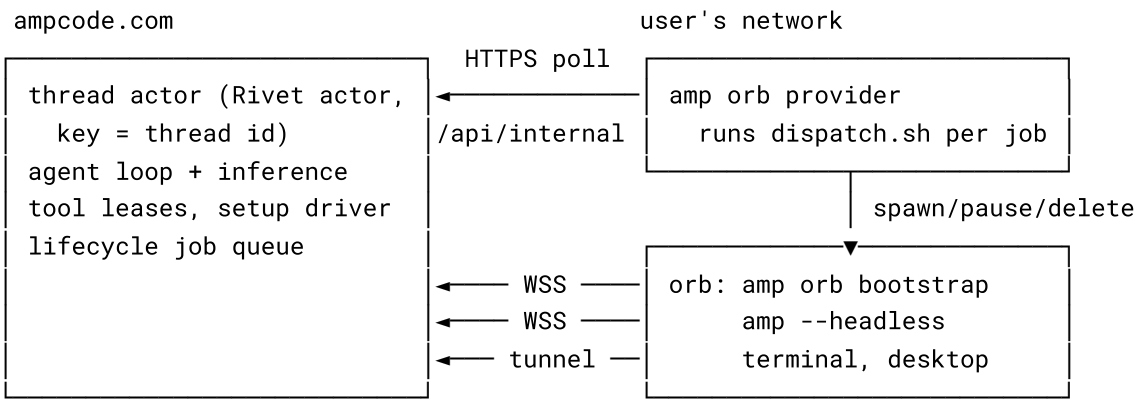
1.1 Vocabulary

term	meaning
thread	One agent conversation. Each thread owns at most one orb.

term	meaning
orb	The machine assigned to a thread. Named `ORB_ID`, e.g. `amp-<thread-uuid>` or `moonlet-<id>`.
control plane	The server that stores threads, calls the model and decides when orbs run. Amp's is ampcode.com; moonlet's is `moonlet server`.
provider	A process inside the user's network that turns lifecycle jobs into machine operations: `amp orb provider` or `moonlet provider`.
dispatch program	A user-supplied executable the provider runs once per job, configured only through `ORB_*` environment variables (§2.1).
job	One lifecycle operation: `spawn` (create or resume), `pause`, or `delete`.
bootstrap	The first process started inside an orb. It dials the control plane with a per-spawn token.
executor	The in-orb component that actually runs tool calls. In Amp, `amp --headless`; in moonlet, `pi-env serve`.
lease	Amp's claim on one tool call by one executor, with a deadline, re-issued after a reconnect.
generation	moonlet's spawn counter for an orb. Tokens carry it; a token from an older generation is refused (§5.3).

2. Amp's Architecture

The decisive design choice is that the agent loop never runs in the orb. Every Amp thread is a Rivet actor on Amp's servers, keyed by the thread ID. The actor holds the transcript, calls the model, schedules tools, drives orb setup and enqueues lifecycle jobs. The orb is a remote executor: it registers tools, accepts leases, runs them and returns results. It holds no model keys and no Amp API key. Pausing an orb therefore never loses agent state, because there is none in the orb to lose.



Listing 1: Amp's components. Every arrow into ampcode.com is an outbound connection from the user's network.

The evidence for this section comes from the processes, files and logs of a live orb on exe.dev, from the CLI binary's string table (its JavaScript is compiled to Bun bytecode), and from pointing `amp orb provider` and `amp orb bootstrap` at fake servers whose deliberately wrong replies made the CLI print the field names it validates. The full report is `docs/amp-orb-internals.md`.

2.1 Provider and Jobs

`amp orb provider` speaks plain HTTPS JSON-RPC: `POST /api/internal?<method>` with body `{"method", "params"}` and the provider token `aop_live_…` as a bearer token. It long-polls for jobs, runs the dispatch program, streams the program's output back as the thread's setup log, and reports completion. It handles several jobs at once and updates itself, draining jobs before it restarts.

method	**purpose**
`orbSpawnerClaimJob`	Long poll. Returns `{claim: null}` or a claim with `id`, `attempt`, `timeoutMs`, `claimDeadlineAt` and the `job`.
`orbSpawnerHeartbeatJob`	Extends the claim while the dispatch program runs.
`orbSpawnerCompleteJob`	Reports the job finished.
`orbSpawnerReleaseClaim`	Gives claims back on shutdown.
`orbSpawnerWaitLogRead`, `orbSpawnerSendLogChunk`	Stream dispatch output to a thread that is watching its setup log.

The job itself is small: `op`, a `sandboxId` of the form `orb-provider:<config>:<orbId>`, a per-spawn `token`, an optional `snapshotId`, and an optional server URL. The provider translates it into the environment of the dispatch program, which is the entire contract between Amp and the user's infrastructure:

variable	value
ORB_OPERATION	spawn, pause or delete.
ORB_ID	Machine name; repeated jobs for one orb carry the same ID.
ORB_IMAGE	Image to boot; the provider's override or Amp's public orb image.
ORB_SNAPSHOT_ID	Project snapshot to boot from (spawn) or to save (pause), §2.5.
ORB_TOKEN	Spawn only. Short-lived credential for this spawn's bootstrap.
AMP_URL	Spawn only, when the server is not ampcode.com.

A spawn must make the machine run and start `amp orb bootstrap` inside it as the image's default user, detached. A pause stops the machine and keeps its disk. A delete removes it. Every operation must be safe to repeat and finish within five minutes.

Failure handling is weak at this layer. A dispatch program that exits non-zero still shows `✓ done` in the provider's output; its stderr becomes the thread's error line; and Amp re-issues the spawn only after two to three minutes. A spawn that exits zero but whose bootstrap never connects is indistinguishable from a slow one until Amp gives up.

2.2 Bootstrap

The bootstrap exchanges its token for a location: `GET /api/orbs/bootstrap` with `Authorization: Bearer $ORB_TOKEN` returns `{threadId, sandboxId, rivetEndpoint, expiresAt}`. It then opens a WebSocket to the thread actor through Rivet's actor gateway:

```
GET {rivetEndpoint}/gateway/threadActor/websocket/?rvt-namespace=default&rvt-method=get&rvt-key=<threadId>
Sec-WebSocket-Protocol: rivet, rivet_encoding.bare, rivet_conn_params.
{"orbToken":…,"transport":"json-rpc"}
```

In production `rivetEndpoint` is a path under ampcode.com carrying a Rivet publishable key. Frames use Rivet's BARE envelope with JSON-RPC inside. Once connected, the bootstrap announces the sandbox and serves a small remote-exec and file API to the actor: `run`, `write_files`, `read_file`, `file_exists`, `remove_file`, `append_file`, `make_directory` and `truncate`. In the observed session the actor issued thirty such requests in five seconds to set the orb up (§2.4). A PID file keeps one bootstrap per machine, so on resume the dispatcher must kill the previous one or the new token is never used.

2.3 The Executor

Tools do not run in the bootstrap. The actor uses the bootstrap to write `~/.env` and a supervisor script, and the supervisor loops forever starting `amp --headless=<thread>`. That process is a second Rivet

client of the same actor. It authenticates with thread identity rather than the spawn token, which is why `ORB_TOKEN` never appears in `~/.env`.

```
actor → orb   tool_lease               {toolCallId, toolName, …args}
orb   → actor executor_tool_lease_ack  {toolCallId}
orb   → actor tool_progress            (streamed snapshots)
orb   → actor executor_tool_result     {run: {status, result: {output, exitCode}},
workspaceChanged, timings}
actor → orb   executor_tool_result_ack {toolCallId}
```

Listing 2: One tool call between the thread actor and the headless executor.

The handshake before the first lease sends `executor_connect` with capability tags, an environment snapshot (repository, platform, working directory), the guidance files and skills found in the workspace, and `executor_tools_register` with twenty-two tool names: `shell_command` and its async variants, `Read`, `edit_file`, `apply_patch`, `create_file`, `glob`, `Grep`, `web_search`, thread file transfer, and plugin and skill reloads. The model sees more tools than the orb registers because some run on the server. The actor also runs git directly through `executor_git_command` and `executor_git_diff_snapshot`, which is what fills the Changes view. Because a lease has a deadline, a call whose executor disappears is released after reconnect (`client_resume`).

A third process, the plugin runtime, is a Bun child speaking stdio JSON-RPC that contributes tools, skills and agent modes from installed plugins.

2.4 Setup

Setup is driven entirely from the actor through the bootstrap's `run` and `write_files`:

step	what happens
Git	Clears old credential helpers; installs a helper that serves a bootstrap GitHub token for the clone, then `amp git-credential-helper` for short-lived tokens; rewrites SSH remotes to HTTPS; configures SSH commit signing through `amp-sign-commit`; installs a `gh` wrapper.
Clone	Clones the project into `/home/user/workspace/repo`, unless the provider disables cloning for images that bring their own checkout.
Runtime	Writes `~/.env` and the headless supervisor; waits for `executor_connected`.
Hooks	Runs the project's pre-setup script and `.agents/setup` (or `.agents/resume` on resume), logging through `amp orb redact-output`. A successful setup is recorded and makes a project snapshot possible.
Services	Reconciles `.amp/services.yaml` into supervised processes, each with a portal URL.

2.5 Lifecycle

Idle detection happens on the server: the actor knows when the agent has finished and how many observers are attached, and sends `pause` a few minutes after the thread goes quiet. The next message sends `spawn` again with a fresh token. On resume, a new bootstrap starts, the `resume` hook runs, the executor reconnects with `resumeBootstrap`, and unfinished leases are re-issued.

A project snapshot is a paused orb with the repository cloned and `.agents/setup` already run. Amp saves one by sending `pause` with `ORB_SNAPSHOT_ID` followed by `spawn` for the same orb; the dispatcher stores a copy of the disk under that ID on pause. Later threads of the same project carry the ID on spawn, and a dispatcher that has the copy boots from it and skips setup. A dispatcher that ignores the variable merely runs setup every time.

3. What Orbs Support

The machine is the agent's, but several features let a human share it. All of them reach the orb through outbound connections; nothing in the orb listens on a public interface.

feature	mechanism
Changes view, Ship	The actor runs `git status` and diff snapshots through the executor; pushes use the git credential helper (§2.4).
Terminal	A shared tmux session behind a loopback terminal server, reached through Amp Tunnel (`tunnel.ampcode.com`). The agent and the human see the same session.
Portals	Services from `.amp/services.yaml` or `amp orb service start`, supervised and restarted after resume, each exposed at a URL that only signed-in users with access to the thread can open.
Desktop	A Wayland desktop with Chrome; `amp orb desktop ensure` prints a DevTools endpoint for browser automation.
Files	Thread attachments, drag-and-drop into the file pane or terminal (up to 512 MB), and `amp sync` to mirror changes to a local checkout.
Amp Net	Routes an orb's outbound traffic through an exit node on the user's own machine.
Secrets	Workspace and project secrets written into `~/.env`, with output redaction by value.
Snapshots	Project snapshots skip setup for new threads (§2.5).
Sizes	Hosted orbs come in sizes from `a1.tiny` (1 CPU, 2 GB) to `a1.xxlarge`; self-hosted sizes are whatever the dispatch program creates.

4. Self-Hosting on exe.dev

exe.dev provides VMs that start in seconds, pause with their RAM snapshotted to disk, and resume in a few seconds, which meets Amp's requirements directly. The CLI is `ssh exe.dev <command>`; each VM is reachable as `ssh <name>.exe.xyz`.

The setup required an Amp workspace (individual accounts have none until one is created; the workspace `swairshah` now holds the account's credits and shared threads), a custom orb provider named `exe-dev` registered under Settings → Orbs, and the provider process running locally with `exe-provider/dispatch.sh`. A thread selects the provider under Orb → Self-Hosted.

```
if [[ -z "$status" ]]; then
  ssh exe.dev new --name="$ORB_ID" --image="$ORB_IMAGE" --cpu=2 --memory=8G --no-email
elif [[ "$status" == paused ]]; then
  ssh exe.dev resume "$ORB_ID"
fi
for _ in $(seq 1 30); do ssh "$ORB_ID.exe.xyz" true && break; sleep 2; done
{ printf 'export ORB_TOKEN=%q\n' "$ORB_TOKEN"; cat <<'EOF'
[ -e /dev/fd ] || ln -s /proc/self/fd /dev/fd
sudo -u user --preserve-env=ORB_TOKEN,AMP_URL bash -lc \
  'cd; setsid -f amp orb bootstrap </dev/null >>.amp-bootstrap.log 2>&1'
EOF
} | ssh "$ORB_ID.exe.xyz" bash -s
```

Listing 3: The spawn branch of the exe.dev dispatch program, condensed.

The first two spawns failed, and the third worked. Each failure taught something about the platform:

problem	consequence	fix
`ssh exe.dev new` splits its arguments on spaces.	`--comment="Amp orb"` became a stray positional argument and creation failed.	No flag value may contain a space.
A fresh VM drops the first SSH session while it boots.	The VM existed but the bootstrap never started; Amp waited about three minutes.	Retry `ssh … true` until it succeeds; start the bootstrap with `setsid -f`.
exe.dev guests have no `/dev/fd`.	Amp's setup hook redirects through `> (redact_output)` and dies, so setup is never cached and no snapshot is ever taken.	Link `/proc/self/fd` to `/dev/fd` before the bootstrap starts.
The default image is listed as `boldsoftware/exeuntu` but is not pullable by that name.	Creation fails when it is passed to `--image`.	Omit `--image` for the default image.

problem	consequence	fix
The Individual plan allows 50 VMs and 4 vCPUs and 16 GB in total.	Orb creation fails at the cap; paused orbs still count.	Delete finished orbs; size orbs small.

Creating a VM from Amp's orb image took 48 seconds, most of it pulling the image. Once running, the Amp smoke-test thread executed in a Debian 12 orb with two CPUs and 7.7 GB of memory. Amp had not sent a pause after ninety minutes with a browser tab still open on the thread, consistent with observer-based idle detection (§2.5).

5. Moonlet

moonlet keeps Amp's shape and replaces every proprietary component. The control plane is a pi-durable Harness; the executor is pi-env's daemon; the provisioning contract is unchanged, so the exe.dev dispatch program above drives both systems.

Amp	moonlet
Rivet thread actor: transcript, inference, leases	pi-durable `Harness` over SQLite; one conversation per thread; tool calls are durable tasks
`/api/internal?orbSpawner*`	`POST /api/provider/claim` (long poll) and `POST /api/provider/jobs/:id`
`amp orb provider`	`moonlet provider`, or the same loop inside the server with `--dispatch`
`amp orb bootstrap` + Rivet WebSocket	`moonlet bootstrap`: one outbound control socket, one outbound socket per pipe
`amp --headless` executor + tool leases	`pi-env serve`, reached by pi-env's own `RemoteExecutionEnv`
Setup through `run/write_files`	A setup script run through the same `ExecutionEnv` (§5.4)

```
user — CLI / UI ─► moonlet server
                   ├─ pi-durable Harness
                   │    └─ env() ─► RemoteExecutionEnv ─► pi-env Connection
                   │                                          │ spawns
                   │                                    moonlet relay
                   │                                          │ loopback WS
                   ├─ orb manager ◄──────────────── splice ──┘
                   │    ▲ control WS          ▲ pipe WS
                   │    └── orb: moonlet bootstrap — stdio — pi-env serve
                   └─ job queue ─► dispatch program ─► exe.dev, local, …
```

Listing 4: moonlet's components. The orb only dials out; the control plane splices pipes it never parses.

5.1 Control Plane

Each thread is a row pointing at a pi-durable conversation and an orb ID. A message is committed to the conversation the moment it arrives; the Harness then runs the turn as durable tasks: a generation task calls the model and owns one tool task per call. The bridge to the orb is pi-durable's `env` option, a function the Harness calls whenever it needs a conversation's execution environment: to render the system prompt and to run each tool.

```ts
env: async ({ conversationId, cwd }) => {
  const thread = store.threadByConversation(Number(conversationId));
  if (!thread || !threads) return undefined;
  const workspace = await threads.prepare(thread); // spawn, connect, clone, .agents/setup
  return new RemoteExecutionEnv({
    connection: orbs.connection(thread.orb_id),
    id: `moonlet:${thread.orb_id}`,
    cwd: cwd ?? workspace,
  });
},
```

Listing 5: The env factory: the first use of a thread's environment starts its orb.

The registry installs pi-durable's `CodingTools` (`read`, `write`, `edit`, `bash`) and one extension whose prompt section tells the model it is in a pausable remote machine. Because tools receive their environment from the factory, none of them knows the orb exists. Because the conversation is durable, a server restart mid-turn resumes the run, and a tool call cut off by a lost connection returns an `interrupted` error to the model, as pi-durable does for any crash.

5.2 The Pipe

pi-env already solves remote execution for pi-durable: a small Rust daemon, `pi-env serve`, performs file operations and commands on the remote machine and speaks a framed binary protocol over stdin and stdout, and `RemoteExecutionEnv` implements the full `ExecutionEnv` interface against it with results identical to a local `NodeExecutionEnv`. Its `Connection` normally starts the daemon through `ssh`. Orbs,

however, accept no inbound connections, so moonlet substitutes the transport without touching the protocol.

`Connection` accepts an async `command` function and spawns whatever argv it returns, appending `serve --token <hex>`. moonlet returns the argv of its own `relay` subcommand, after first making sure the orb is connected. The 60-second start timeout of `Connection` begins only after the function returns, so waking or creating an orb inside it is safe.

step	actor	action
1	pi-env `Connection`	Calls `command()`; moonlet awaits `ensureRunning(orb)`, then returns `[moonlet, relay, <orb>]`.
2	`moonlet relay`	Opens `/internal/relay/<orb>` on loopback with a per-process key, sending the daemon arguments.
3	orb manager	Allocates a pipe ID and sends `open-pipe {pipeId, args}` on the orb's control socket.
4	`moonlet bootstrap`	Starts `pi-env serve --token <hex>` and dials `/orb/pipe/<pipeId>` with its orb token.
5	orb manager	Splices the two sockets, flushing bytes buffered while the orb connected.

From then on pi-env's frames, pings and cancellations flow end to end. moonlet inspects only frame sizes: pi-env pings are fifteen bytes, so any larger message counts as activity for idle detection. When either side closes, the other is closed; pi-env's daemon kills its process groups and exits when its stdin ends, so a dropped pipe cannot leave commands running.

5.3 Orb Lifecycle

The orb manager keeps one row per orb with a status (`none`, `spawning`, `running`, `pausing`, `paused`, `deleting`, `deleted`, `error`) and a generation counter, plus in-memory control sockets, pipes and pi-env connections.

`ensureRunning`	Returns at once if the orb's control socket has said hello. Otherwise it waits for any queued job of the orb, increments the generation, signs a token for `(orb, generation, expiry)`, queues a spawn job, and resolves when the bootstrap connects. A dispatch failure rejects immediately with the program's output; a failed spawn is remembered for fifteen seconds so the many environment calls of one turn do not each retry it.
`pause`	Closes the orb's pi-env connection, tells the bootstrap to shut down (close code 4409, which it treats as final), and queues a pause job. A timer pauses every orb that has no run in progress and no activity for `--idle-minutes`.

`delete`	The same, with a delete job, when the thread is removed.

A spawn counts as successful only when the bootstrap connects, not when the dispatch program exits zero (§2.1). Tokens are HMAC-signed and bound to a generation, so after a resume the previous bootstrap, frozen in the RAM snapshot and thawed with the VM, is refused when it tries to reconnect and exits. Jobs are claimed one orb at a time in order, so a resume cannot overtake the pause before it; jobs claimed by a provider that vanished are re-queued after ten minutes; the spawn token is removed from a job row once the job finishes.

5.4 Setup

The first time a thread's environment is needed, `prepare` runs once per orb: it waits for the orb, asks pi-env for the remote home directory, clones the thread's repository into `~/workspace/repo` if there is one, runs `.agents/setup` if present, sets the conversation's working directory, and appends a `moonlet.setup` entry with the exit code and output. The entry kind is not `pi.*`, so the user sees it and the model does not. A failed setup is recorded but does not block the thread, as in Amp. An optional `MOONLET_GIT_TOKEN` is passed to the clone as an HTTP header and never written into the orb's git configuration.

5.5 Dispatch

moonlet passes the same variables as Amp (§2.1) plus two: `ORB_SERVER_URL`, the control plane's public URL, and `ORB_BOOTSTRAP`, the shell command to run in the orb. A dispatch program that runs `${ORB_BOOTSTRAP:-amp orb bootstrap}` therefore serves both systems, which is how `dispatch/exe-dev.sh` works. moonlet's command fetches `/orb/install.sh`, which downloads the `moonlet` and `pi-env` binaries for the machine's architecture, verifies their SHA-256 against a manifest, caches them in `~/.moonlet/bin`, and starts the bootstrap with `setsid -f`. An image needs only bash, curl and coreutils; the default exe.dev image works unmodified. `dispatch/local.sh` runs orbs as local processes with their own home directories, which exercises every path except the VM.

5.6 Security

boundary	rule
Orb → control plane	Outbound WebSockets only. Tokens travel in `Sec-WebSocket-Protocol` because WebSocket clients cannot set headers; they are signed, expiring, and bound to orb and generation.
Model credentials, transcripts	Stay on the control plane. The orb sees file contents and command output, as its tools require.
API	Bearer `apiToken`; providers use a separate `providerToken`; both generated once into `config.json` with mode 600.

boundary	rule
Relay endpoint	Loopback only, rejecting requests that carry proxy headers (a tunnel on the same host also connects from loopback), plus a per-process key.
Downloads	`/orb/install.sh` and the binaries are public; they are open source and verified by hash.

6. Results

Both systems were exercised end to end against the same exe.dev account. Times are wall clock from the control plane's log.

path	measurement
moonlet, local dispatch	Spawn, connect and setup under one second; the model's `bash` call ran in the local orb; idle pause after two minutes; a follow-up resumed the orb as generation 2 with its files.
moonlet, exe.dev create	VM created in 5 s; bootstrap downloaded (a 100 MB Bun binary) and connected 26 s later; clone and setup finished 32 s after the spawn.
moonlet, exe.dev pause	Paused by the idle timer after 5 min in 2.9 s, RAM snapshotted.
moonlet, exe.dev resume	7 s from the spawn request to the bootstrap's hello; the repository and an uptime continuing across the pause confirmed a resume rather than a reboot.
Amp, exe.dev	48 s to create from Amp's image; thirty setup requests in five seconds; smoke test passed in a Debian 12 orb.

The failures were as instructive as the successes. In moonlet, a dispatch error became a tool error result that the model reported in plain language, and the thread recovered on the next message without intervention, which is the behavior durable tool tasks were designed for.

7. Open Questions

Project snapshots are the largest missing piece. On exe.dev, `ssh exe.dev cp` can copy a prepared VM, but a copy is a running VM that counts against the plan's VM cap, so the natural design (pause, copy, pause the copy) needs a cheaper primitive or a different provider.

The terminal and portals have a clear path: pi-env already exposes a pseudo-terminal through `openTerminal(connection)`, so a terminal can ride the same pipe as tools, and portals can use the same splicing with a byte stream to a port instead of to `pi-env serve`. A Changes view needs only `git status` and `git diff` through the existing environment.

Idle detection in moonlet is local to one process: it counts runs started through the API and non-ping pipe traffic. Runs resumed after a control-plane restart are not counted until the next message, so an orb could be paused during one; the run would then resume the orb on its next tool call, at the cost of a lost process.

The bootstrap binary is 100 MB because it carries the Bun runtime. A bootstrap written in a compiled language, or in pi-env's Rust next to the daemon, would shrink every first spawn by most of its download time.

Finally, both providers currently run in terminal tabs. Amp's documentation asks for a service manager so the provider survives reboots; the same applies to `moonlet server` and its tunnel, and a deployed control plane on a public host would replace the tunnel entirely.